Documentation de la base de données

2026

INTRODUCTION

Ce document présente la conception et la structure de la base de données du projet **CoffreFort**, développé dans le cadre du BTS SIO.
La base de données a pour objectif de supporter les fonctionnalités principales d'un coffre-fort numérique : gestion des utilisateurs, organisation hiérarchique des dossiers, stockage et versionnage des fichiers, partage sécurisé via lien public et journalisation des téléchargements.
Ce document détaille l'architecture relationnelle, les contraintes d'intégrité, les choix techniques réalisés ainsi que les règles de gestion métier associées.

1. VUE D'ENSEMBLE

1.1 Description générale

La base de données **CoffreFort** est une base relationnelle conçue sous MySQL (moteur InnoDB). Elle centralise les métadonnées nécessaires au fonctionnement de l'application.
Les fichiers physiques sont stockés sur le système de fichiers, tandis que la base de données conserve les informations structurelles, de sécurité et de traçabilité.

1.2 Besoins couverts

La base permet de :

- Gérer les utilisateurs (authentification, rôles administrateur, gestion des quotas).
- Organiser les ressources via une arborescence de dossiers hiérarchiques.
- Stocker les métadonnées des fichiers et assurer leur versionnage.
- Mettre en place des partages sécurisés via token.
- Journaliser les téléchargements afin d'assurer la traçabilité.

1.3 Objectifs techniques

- Garantir l'intégrité des données via l'utilisation de clés étrangères.
- Assurer la cohérence des relations entre entités.
- Optimiser les performances grâce à des index adaptés.
- Permettre l'évolution future du système.

2. ARCHITECTURE DE LA BASE DE DONNÉES

2.1 Entités principales

La base repose sur les entités suivantes :

- ***user :*** comptes utilisateurs de l'application.
- ***folder :*** dossiers organisés en arborescence.

- ***file :*** métadonnées des fichiers.
- ***file_version*** *:* gestion du versionnage des fichiers.
- ***share*** *:* liens de partage publics.
- ***downloads_log*** *:* journalisation des téléchargements.
- ***audit_log*** : journalisation des actions métier.

2.2 Relations principales

- Un utilisateur possède plusieurs dossiers, fichiers et partages.
- Un dossier peut contenir des sous-dossiers (relation récursive) ainsi que des fichiers.
- Un fichier possède une ou plusieurs versions.
- Un partage appartient à un utilisateur et cible une ressource (fichier ou dossier).
- Un téléchargement peut être associé à un partage et/ou à une version précise.

2.3 Choix techniques de modélisation

- Utilisation du moteur InnoDB afin de bénéficier des contraintes d'intégrité référentielle.
- Utilisation de types BIGINT UNSIGNED pour anticiper une montée en charge.
- Mise en place de suppressions en cascade lorsque cela est pertinent afin de garantir la cohérence des données.
- Utilisation d'une association polymorphe dans la table *shares* via les champs (kind, target_id) pour cibler soit un fichier, soit un dossier. Ce choix simplifie le modèle dans le cadre du MVP et pourra être amélioré dans une version future.

3. DESCRIPTION DES TABLES

3.1 Table user

Cette table contient les comptes utilisateurs.

Champs principaux :

- id : identifiant unique.
- email : adresse email (unique).
- pass_hash : mot de passe haché.
- quota_total : quota total attribué.
- quota_used : espace actuellement utilisé.
- is_admin : indique si l'utilisateur possède des droits administrateur.
- created_at : date de création du compte.

Contraintes :

- Unicité de l'email.
- Gestion des quotas assurée par la logique applicative.

3.2 Table folder

Permet l'organisation hiérarchique des dossiers.

Champs :

- id : identifiant unique.
- user_id : référence vers l'utilisateur propriétaire.
- parent_id : référence vers un dossier parent (nullable).
- name : nom du dossier.
- created_at : date de création du compte.

Contraintes :

- Clé étrangère vers users(id) avec suppression en cascade.
- Clé étrangère récursive vers folders(id) avec suppression en cascade.
- Index sur user_id.

3.3 Table file

Contient les métadonnées des fichiers.

Champs :

- id : identifiant unique.
- user_id : propriétaire du fichier.
- folder_id : dossier parent (nullable).
- original_name : nom d'origine.
- stored_name : nom utilisé pour le stockage physique.
- mime : type MIME.
- size : taille du fichier.
- created_at, updated_at : dates de suivi.

Contraintes :

- Suppression en cascade si l'utilisateur est supprimé.
- folder_id défini en ON DELETE SET NULL.
- Index composite (user_id, folder_id).

3.4 Table file_version

Gère le versionnage des fichiers.

Champs :

- id : identifiant unique.
- file_id : référence vers le fichier.
- version : numéro de version.
- stored_name : nom de stockage.
- iv : vecteur d'initialisation.
- auth_tag : tag d'authentification.
- key_envelope : clé chiffrée.
- checksum : empreinte de contrôle.
- size
- created_at

Contraintes :

- Clé étrangère vers files(id) avec suppression en cascade.
- Contrainte d'unicité (file_id, version).

Aspect sécurité

- Le chiffrement repose sur l'algorithme AES-GCM.
- Le stockage du iv et du auth_tag permet de garantir l'intégrité et l'authenticité des données chiffrées.
- Le checksum renforce la vérification d'intégrité.

3.5 Table share

Permet la création de liens de partage sécurisés.

Champs :

- id : identifiant unique.
- user_id
- kind : type de ressource (file ou folder).
- target_id : identifiant de la ressource ciblée.
- token : identifiant unique du lien.
- token_sig : signature du token.
- expires_at
- max_uses
- remaining_uses
- is_revoked
- allow_fixed_versions
- created_at

Contraintes :

- Unicité du token.
- Clé étrangère vers users(id).

3.6 Table downloads_log

Journalise les téléchargements.

Champs :

- id : identifiant unique.
- share_id
- version_id
- downloaded_at
- ip
- user_agent
- success
- message

Contraintes :

- Clé étrangère vers shares(id) avec suppression en cascade.
- Clé étrangère vers file_versions(id) avec ON DELETE SET NULL.
- Index sur share_id.

3.7 Table audit_log (optionnelle)

Permet la journalisation des actions applicatives.
Elle enregistre notamment :

- l'utilisateur concerné,
- le type d'action réalisée,
- la table impactée,
- les informations complémentaires,
- l'adresse IP et le user-agent.

Des index sont présents sur user_id, action, created_at et (table_name, record_id) afin d'optimiser les recherches.

4. INDEX ET INTÉGRITÉ

Les contraintes mises en place garantissent :

- L'intégrité référentielle entre les entités.
- L'unicité des identifiants critiques (email, token, version).
- Des performances optimisées sur les recherches fréquentes grâce aux index ciblés.

5. RÈGLES DE GESTION MÉTIER

Utilisateur

- Chaque utilisateur dispose d'un quota de stockage.
- Les administrateurs peuvent gérer les comptes et les quotas.

Dossier

- Un dossier appartient à un utilisateur.
- La suppression d'un dossier entraîne la suppression de ses sous-dossiers.

Fichier

- Un fichier appartient à un utilisateur et peut être rattaché à un dossier.
- Toute modification crée une nouvelle version.

Version

- Une version est immuable.
- La version la plus récente est considérée comme version courante.

Partage

- Un partage peut expirer ou être limité en nombre d'utilisations.

- Un partage peut être révoqué à tout moment.
- L'option allow_fixed_versions autorise ou non le téléchargement d'une version spécifique.

Journalisation

- Les téléchargements sont enregistrés afin d'assurer la traçabilité.

6. MAINTENANCE ET BONNES PRATIQUES

- Sauvegardes régulières de la base de données.
- Vérification de cohérence entre fichiers physiques et enregistrements BDD.
- Surveillance des quotas.
- Journalisation des actions sensibles.

7. ÉVOLUTIONS FUTURES

- Séparation stricte des partages fichiers/dossiers pour améliorer la modélisation.
- Ajout de vues analytiques.
- Renforcement des mécanismes de sécurité (ex : authentification forte).
- Ajout de contraintes d'unicité supplémentaires sur certains champs métiers.

CONCLUSION

La base de données CoffreFort a été conçue afin de garantir l'intégrité, la sécurité et la traçabilité des données.
Elle répond aux exigences fonctionnelles du projet tout en respectant les bonnes pratiques de conception relationnelle.
Sa structure permet une évolution future vers une solution plus complète et scalable.